

# 编译原理语法分析知识点文档

## 从 CFG、LL 到 LR 族与错误恢复

面向复习与系统掌握的中文技术笔记

2026 年 6 月 27 日

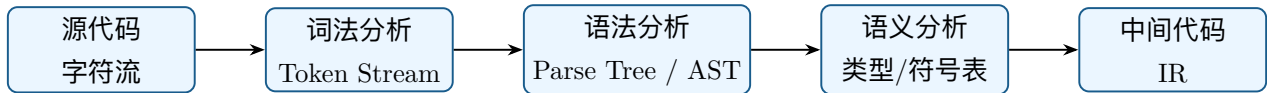
### 目录

|                      |   |
|----------------------|---|
| 阅读导引                 | 3 |
| 1 语法分析基本概念           | 3 |
| 1.1 语法分析器的职责         | 3 |
| 1.2 分析方法分类           | 4 |
| 2 上下文无关文法 CFG        | 4 |
| 2.1 CFG 四元组          | 4 |
| 2.2 推导、句型、句子与语言      | 5 |
| 2.3 分析树与 AST         | 5 |
| 2.4 二义性与消歧           | 6 |
| 3 文法变换与 FIRST/FOLLOW | 6 |
| 3.1 左递归              | 6 |
| 3.2 左因子提取            | 7 |
| 3.3 FIRST 集          | 7 |
| 3.4 FOLLOW 集         | 7 |
| 4 自上而下分析与 LL         | 8 |
| 4.1 自上而下分析思想         | 8 |
| 4.2 递归下降分析           | 8 |
| 4.3 LL(1) 文法条件       | 9 |
| 4.4 预测分析表构造          | 9 |

|                               |           |
|-------------------------------|-----------|
| 目录                            | 2         |
| 4.5 非递归 LL(1) 分析器             | 9         |
| <b>5 自下而上分析与移进-规约</b>         | <b>10</b> |
| 5.1 自下而上分析思想                  | 10        |
| 5.2 短语、直接短语、句柄                | 10        |
| 5.3 移进-规约动作                   | 10        |
| 5.4 冲突                        | 11        |
| <b>6 LR 分析通用框架</b>            | <b>11</b> |
| 6.1 LR(k) 含义                  | 11        |
| 6.2 LR 分析器结构                  | 11        |
| <b>7 LR(0)、SLR、LR(1)、LALR</b> | <b>12</b> |
| 7.1 LR(0) 项与项目集族              | 12        |
| 7.2 CLOSURE 与 GOTO            | 12        |
| 7.3 LR(0) 表构造                 | 13        |
| 7.4 SLR(1)                    | 13        |
| 7.5 LR(1)                     | 14        |
| 7.6 LALR(1)                   | 14        |
| <b>8 LL 与 LR 对比</b>           | <b>15</b> |
| <b>9 语法错误恢复</b>               | <b>15</b> |
| 9.1 错误恢复目标                    | 15        |
| 9.2 常见错误恢复策略                  | 16        |
| 9.3 Panic-mode 流程             | 16        |
| 9.4 LL 与 LR 中的错误恢复            | 16        |
| <b>10 综合示例：表达式文法从 LL 到 LR</b> | <b>16</b> |
| <b>11 复习清单</b>                | <b>17</b> |
| <b>结语</b>                     | <b>18</b> |

## 阅读导引

语法分析是编译器前端的核心阶段。词法分析器把字符流切成 token，语法分析器根据语言文法判断 token 序列是否构成合法程序结构，并输出 parse tree 或 AST，供语义分析和中间代码生成使用。



---

### 总览

语法分析的主线是：用上下文无关文法描述语言结构；用自上而下或自下而上的算法识别句子；用 LL/LR 表驱动方法把分析过程机械化；用错误恢复机制在遇到语法错误时尽量继续分析。

---

## 1 语法分析基本概念

### 1.1 语法分析器的职责

---

#### 语法分析

语法分析（syntax analysis / parsing）读取 token 序列，依据上下文无关文法判断其结构是否合法，并构造语法结构表示，通常是 parse tree 或 AST。

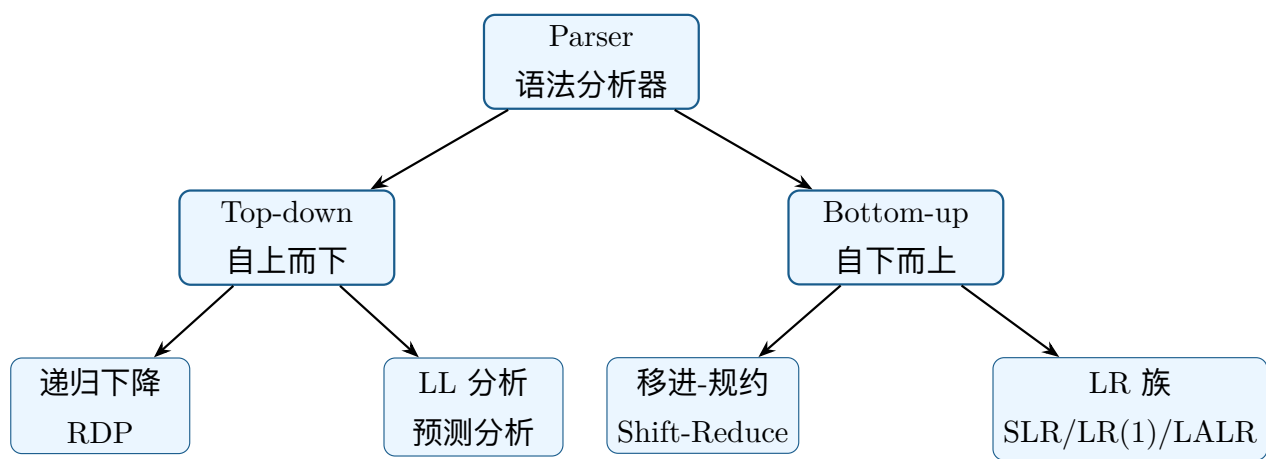
---

语法分析器主要完成：

- 验证 token 序列是否符合语言文法。
- 在合法时构造程序结构，如表达式、语句、声明、函数体。
- 在非法时报告语法错误，并尽可能恢复继续分析。
- 为后续语义分析提供结构化输入。

词法分析只能识别局部模式，例如标识符、数字、关键字；语法分析处理嵌套和递归结构，例如括号匹配、表达式优先级、语句块、函数定义。

1.2 分析方法分类



| 类别   | 思路               | 典型算法                              |
|------|------------------|-----------------------------------|
| 自上而下 | 从开始符号出发，尝试推导出输入串 | 递归下降、LL(1)、LL(k)                  |
| 自下而上 | 从输入串出发，逐步规约回开始符号 | shift-reduce、LR(0)、SLR、LR(1)、LALR |

2 上下文无关文法 CFG

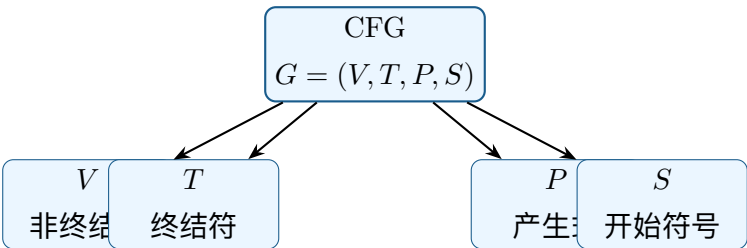
2.1 CFG 四元组

上下文无关文法

上下文无关文法 CFG 是四元组

$$G = (V, T, P, S).$$

$V$  是非终结符集合,  $T$  是终结符集合,  $P$  是产生式集合,  $S$  是开始符号。



表达式文法示例：

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

其中  $E, T, F$  是非终结符,  $id, +, *, (, )$  是终结符。

## 2.2 推导、句型、句子与语言

- 推导：反复用产生式右部替换左部非终结符的过程。
- 句型：从开始符号可推导出的任意符号串，可包含非终结符。
- 句子：只含终结符的句型。
- 文法语言：所有可由开始符号推导出的句子集合。

最左推导每次替换最左非终结符；最右推导每次替换最右非终结符。自上而下分析常对应最左推导；自下而上分析常对应最右推导的逆过程。

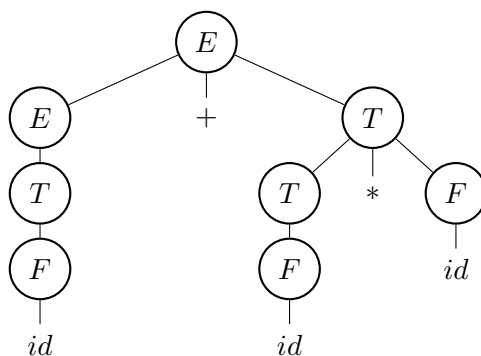
## 2.3 分析树与 AST

---

### Parse Tree

分析树把推导过程表示成树：根是开始符号，内部节点是非终结符，叶子从左到右组成输入串。

---




---

### AST

抽象语法树 AST 去掉多余语法细节，只保留程序语义结构。例如括号节点、单一产生式链条通常会被省略。

---

Parse tree 更适合证明输入符合文法；AST 更适合后续语义分析和代码生成。

## 2.4 二义性与消歧

### 二义性

若某个句子在同一文法下有两棵不同分析树，或有两个不同最左/最右推导，则该文法是二义的。

典型二义文法：

$$E \rightarrow E + E \mid E * E \mid (E) \mid id.$$

句子  $id + id * id$  既可理解为  $(id + id) * id$ ，也可理解为  $id + (id * id)$ 。

常用消歧方法：

- 重写文法，把优先级编码到非终结符层次中。
- 使用结合性规则，如加法左结合。
- 在 parser generator 中声明优先级和结合性。

表达式的非二义文法常写成：

$$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow (E) \mid id.$$

这里  $T$  比  $E$  更靠近叶子，表示乘法优先级高于加法。

## 3 文法变换与 FIRST/FOLLOW

### 3.1 左递归

#### 左递归

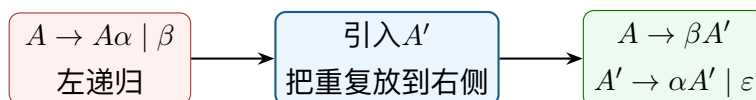
若非终结符  $A$  可推导出以自身开头的句型，即  $A \Rightarrow^+ A\alpha$ ，则  $A$  存在左递归。直接左递归形如  $A \rightarrow A\alpha \mid \beta$ 。

左递归会导致递归下降分析无限递归。直接左递归消除：

$$A \rightarrow A\alpha_1 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \dots \mid \beta_n$$

改写为：

$$\begin{aligned} A &\rightarrow \beta_1 A' \mid \dots \mid \beta_n A' \\ A' &\rightarrow \alpha_1 A' \mid \dots \mid \alpha_m A' \mid \varepsilon. \end{aligned}$$



### 3.2 左因子提取

若多个候选式有公共前缀：

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2,$$

预测分析器看到 $\alpha$  时无法立刻选择。提取左因子：

$$A \rightarrow \alpha A', \quad A' \rightarrow \beta_1 \mid \beta_2.$$

通俗地说：先吃掉共同开头，再根据后续 token 作选择。

### 3.3 FIRST 集

#### FIRST

$\text{FIRST}(\alpha)$  是可能出现在 $\alpha$  推导结果最前面的终结符集合。若 $\alpha \Rightarrow^* \varepsilon$ ，则 $\varepsilon \in \text{FIRST}(\alpha)$ 。

计算规则：

- 若 $X$  是终结符，则 $\text{FIRST}(X) = \{X\}$ 。
- 若 $A \rightarrow \varepsilon$ ，则 $\varepsilon \in \text{FIRST}(A)$ 。
- 若 $A \rightarrow X_1 X_2 \cdots X_n$ ，先把 $\text{FIRST}(X_1) - \{\varepsilon\}$  加入 $\text{FIRST}(A)$ ；若 $X_1$  可为空，再看 $X_2$ ，依此类推。
- 若所有 $X_i$  都可为空，则 $\varepsilon \in \text{FIRST}(A)$ 。

### 3.4 FOLLOW 集

#### FOLLOW

$\text{FOLLOW}(A)$  是在某个句型中可能紧跟在非终结符 $A$  后面的终结符集合。若 $A$  可出现在输入末尾，则 $\$ \in \text{FOLLOW}(A)$ 。

计算规则：

- 对开始符号 $S$ ， $\$ \in \text{FOLLOW}(S)$ 。

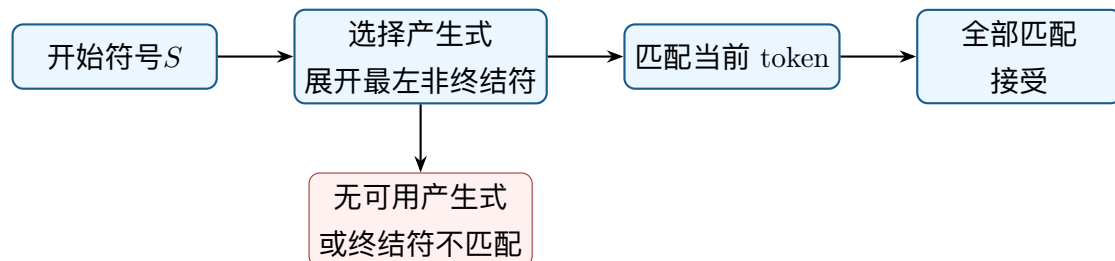
- 若  $A \rightarrow \alpha B \beta$ ，则把  $\text{FIRST}(\beta) - \{\epsilon\}$  加入  $\text{FOLLOW}(B)$ 。
- 若  $A \rightarrow \alpha B \beta$  且  $\beta \Rightarrow^* \epsilon$ ，则把  $\text{FOLLOW}(A)$  加入  $\text{FOLLOW}(B)$ 。
- 若  $A \rightarrow \alpha B$ ，则把  $\text{FOLLOW}(A)$  加入  $\text{FOLLOW}(B)$ 。



## 4 自上而下分析与 LL

### 4.1 自上而下分析思想

自上而下分析从开始符号出发，试图逐步展开非终结符，使叶子序列匹配输入。它模拟最左推导。



### 4.2 递归下降分析

递归下降分析为每个非终结符写一个函数。函数根据 lookahead token 选择产生式，并递归调用其他非终结符函数。

优点：

- 直观，容易手写。
- 错误信息可定制。
- 适合工程语言的部分语法。

缺点：

- 左递归会无限递归。
- 若文法不是预测性的，可能需要回溯。
- 回溯会降低效率，也会让错误定位变差。



### 4.3 LL(1) 文法条件

#### LL(1)

LL(1) 表示从左到右扫描输入 (Left-to-right)，构造最左推导 (Leftmost derivation)，只看 1 个向前看符号。

对同一非终结符的任意两个候选式  $A \rightarrow \alpha \mid \beta$ ，LL(1) 要求：

$$\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset.$$

若  $\varepsilon \in \text{FIRST}(\alpha)$ ，还要：

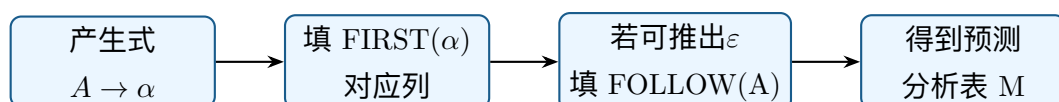
$$\text{FIRST}(\beta) \cap \text{FOLLOW}(A) = \emptyset.$$

直观解释：看一个 token 时，分析器必须唯一知道选哪条产生式。

### 4.4 预测分析表构造

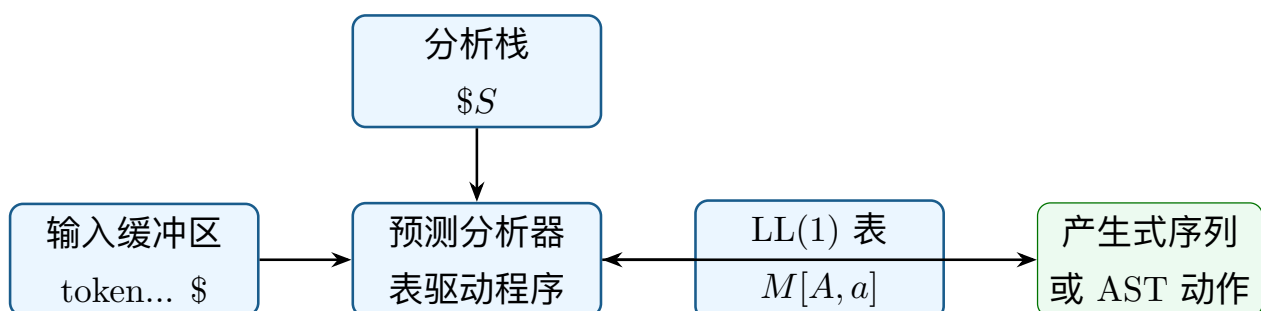
对每条产生式  $A \rightarrow \alpha$ ：

1. 对每个  $a \in \text{FIRST}(\alpha) - \{\varepsilon\}$ ，把  $A \rightarrow \alpha$  填入  $M[A, a]$ 。
2. 若  $\varepsilon \in \text{FIRST}(\alpha)$ ，对每个  $b \in \text{FOLLOW}(A)$ ，把  $A \rightarrow \alpha$  填入  $M[A, b]$ 。
3. 未填的格子表示语法错误。



### 4.5 非递归 LL(1) 分析器

表驱动 LL(1) parser 使用分析栈、输入缓冲区和预测分析表。



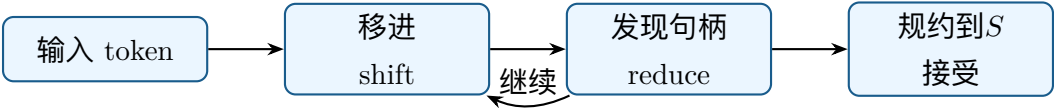
运行规则：

- 若栈顶是终结符且等于当前输入 token，则弹栈并读下一个 token。
- 若栈顶是非终结符  $A$ ，查表  $M[A, a]$ 。
- 若查到  $A \rightarrow X_1 \cdots X_n$ ，弹出  $A$ ，把  $X_n, \dots, X_1$  反向压栈。
- 若查表为空或终结符不匹配，报告错误。

## 5 自下而上分析与移进-规约

### 5.1 自下而上分析思想

自下而上分析从输入串出发，把右部逐步规约成左部，最终规约为开始符号。它对应最右推导的逆过程。



### 5.2 短语、直接短语、句柄

---

#### 句柄

句柄是在某个右句型中，下一步应被规约的产生式右部。把句柄规约为左部，正好逆转最右推导的一步。

---

移进-规约 parser 的任务可以理解为：不断把输入读入栈，在栈顶找到句柄后规约。

### 5.3 移进-规约动作

| 动作     | 含义                                      |
|--------|-----------------------------------------|
| Shift  | 把当前输入符号移入栈，读下一个 token                   |
| Reduce | 若栈顶匹配某产生式右部 $\beta$ ，用左部 $A$ 替换 $\beta$ |
| Accept | 输入已读完，栈中为开始符号，分析成功                      |
| Error  | 没有合法动作，报告语法错误                           |

## 5.4 冲突

---

### Shift-reduce conflict

同一状态、同一 lookahead 下既可以移进, 也可以规约, 称为移进-规约冲突。表达式优先级和 dangling else 常导致此问题。

---

---

### Reduce-reduce conflict

同一状态、同一 lookahead 下可以按两条不同产生式规约, 称为规约-规约冲突。它通常说明文法或语义设计存在更严重的二义性。

---

## 6 LR 分析通用框架

### 6.1 LR(k) 含义

---

#### LR(k)

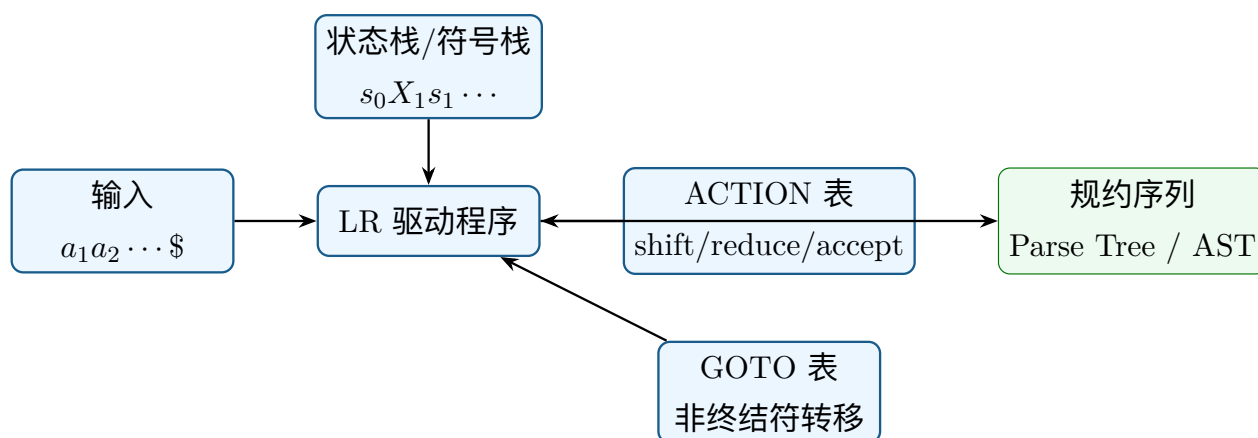
LR(k) 表示从左到右扫描输入 (Left-to-right), 构造最右推导的逆过程 (Rightmost derivation in reverse), 使用  $k$  个 lookahead 符号。

---

LR 分析能力强于 LL, 能处理很多左递归文法, 也更适合自动生成 parser。

### 6.2 LR 分析器结构

LR parser 维护状态栈和符号栈, 依据 ACTION/GOTO 表决定动作。



动作规则：

- $\text{ACTION}[s, a] = \text{shift } t$ : 移进当前输入 $a$ , 压入状态 $t$ 。
- $\text{ACTION}[s, a] = \text{reduce } A \rightarrow \beta$ : 弹出 $|\beta|$  个符号和状态, 再按 $\text{GOTO}[s', A]$  压入新状态。
- $\text{ACTION}[s, \$] = \text{accept}$ : 接受。
- 空表项: 错误。

## 7 LR(0)、SLR、LR(1)、LALR

### 7.1 LR(0) 项与项目集族

#### LR(0) item

LR(0) 项是在产生式右部某位置加点, 例如 $A \rightarrow \alpha \cdot \beta$ 。点表示右部已经识别到的位置。

例子：

$$E \rightarrow E + \cdot T$$

表示已经看到 $E+$ , 接下来希望看到 $T$ 。

### 7.2 CLOSURE 与 GOTO

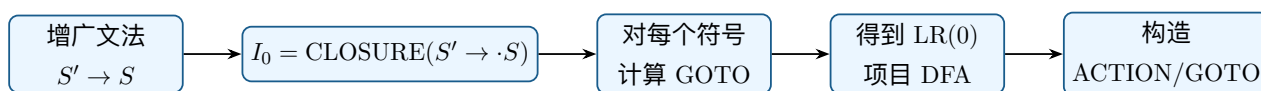
#### CLOSURE

若项目集中有 $A \rightarrow \alpha \cdot B\beta$ , 说明接下来可能识别非终结符 $B$ 。因此要把 $B$  的所有产生式 $B \rightarrow \gamma$  对应项

目  $B \rightarrow \cdot \gamma$  加入集合，直到不再变化。

## GOTO

$GOTO(I, X)$  表示项目集  $I$  在识别符号  $X$  后到达的新项目集。做法是把所有点前为  $X$  的项目点右移一位，再取 CLOSURE。



### 7.3 LR(0) 表构造

对每个项目集状态  $I_i$ ：

- 若  $A \rightarrow \alpha \cdot a\beta$  且  $a$  是终结符， $GOTO(I_i, a) = I_j$ ，则  $ACTION[i, a] = shift\ j$ 。
- 若  $A \rightarrow \alpha \cdot$  是完成项，且  $A \neq S'$ ，则对所有终结符填入  $reduce\ A \rightarrow \alpha$ 。
- 若  $S' \rightarrow S \cdot$ ，则  $ACTION[i, \$] = accept$ 。
- 若  $GOTO(I_i, A) = I_j$  且  $A$  是非终结符，则  $GOTO[i, A] = j$ 。

LR(0) 没有 lookahead，完成项会在所有终结符上规约，因此容易冲突。

### 7.4 SLR(1)

#### SLR(1)

SLR 使用 LR(0) 项目集构造状态，但规约动作只填在  $FOLLOW(A)$  对应列中。它比 LR(0) 更保守，也能减少冲突。

SLR 规约规则：

若状态  $i$  中有完成项  $A \rightarrow \alpha \cdot$ ，则只对

$$a \in FOLLOW(A)$$

填入

$$ACTION[i, a] = reduce\ A \rightarrow \alpha.$$

SLR 的优点是简单；缺点是 FOLLOW 集是全局近似，可能把不该规约的 lookahead 也包含进来，因此分析能力有限。

## 7.5 LR(1)

### LR(1) item

LR(1) 项形如

$$[A \rightarrow \alpha \cdot \beta, a],$$

其中  $a$  是 lookahead。它表示当未来看到  $a$  时，完成的  $A$  才允许规约。

LR(1) 的 Closure 会传播 lookahead：

若有

$$[A \rightarrow \alpha \cdot B\beta, a],$$

则对  $B \rightarrow \gamma$ ，加入

$$[B \rightarrow \cdot \gamma, b] \quad b \in \text{FIRST}(\beta a).$$

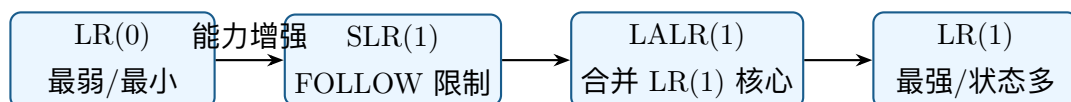
LR(1) 优点是信息精确，分析能力强；缺点是状态数可能很多。

## 7.6 LALR(1)

### LALR(1)

LALR 把 LR(1) 中 LR(0) 核心相同的状态合并，并合并 lookahead 集合。它通常保持接近 LR(1) 的分析能力，同时状态数接近 SLR。

关系图：



更准确地说，文法类包含关系通常为：

$$LR(0) \subset SLR(1) \subset LALR(1) \subset LR(1).$$

状态规模大致为：

$$LR(0) \approx SLR(1) \approx LALR(1) < LR(1).$$

LALR 的风险

合并 LR(1) 状态可能引入 reduce-reduce conflict，但不会引入新的 shift-reduce conflict。LALR 也可能比 LR(1) 更晚发现错误。

8 LL 与 LR 对比

| 维度   | LL             | LR                         |
|------|----------------|----------------------------|
| 方向   | 自上而下，从开始符号推导输入 | 自下而上，从输入规约到开始符号            |
| 推导关系 | 构造最左推导         | 构造最右推导的逆过程                 |
| 典型实现 | 递归下降、预测分析表     | shift-reduce、ACTION/GOTO 表 |
| 文法要求 | 不支持左递归，常需左因子提取 | 可处理很多左递归文法                 |
| 错误定位 | 通常较早、较直观       | 通常能准确知道无法继续的状态             |
| 分析能力 | 较弱，LL(1) 要求严格  | 较强，LR(1) 能处理更大文法类          |
| 工程使用 | 手写 parser 常用   | parser generator 常用        |

直观理解：

- LL 是“先猜结构，再验证输入”。
- LR 是“先读输入，等证据足够后再确认结构”。
- LR 更晚做决定，因此能处理更多文法。

9 语法错误恢复

9.1 错误恢复目标

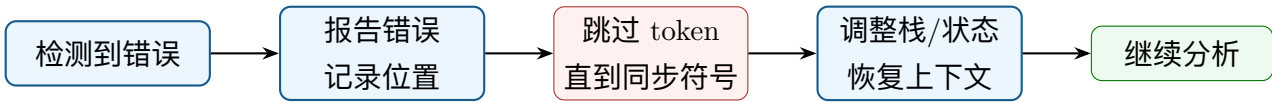
语法分析遇到错误时不能只崩溃退出。好的 parser 应该：

- 报告清晰的位置和原因。
- 避免一处错误引发大量伪错误。
- 尽量跳过错误片段继续分析后续代码。
- 保证恢复过程不会陷入死循环。

9.2 常见错误恢复策略

| 策略                | 做法                     | 特点                |
|-------------------|------------------------|-------------------|
| Panic-mode        | 跳过输入直到同步符号，如分号、右括号、关键字 | 简单稳健，但可能跳过较多内容    |
| Phrase-level      | 在局部插入、删除或替换少量 token    | 错误信息更友好，但规则需谨慎设计  |
| Error productions | 文法中显式加入常见错误产生式         | 能针对常见错误给精确提示      |
| Global correction | 寻找最小编辑距离修复方案           | 理论漂亮，实际代价高，较少用于工程 |

9.3 Panic-mode 流程



常见同步符号：

- 语句结束符：;。
- 块边界：{、}。
- 结构关键字：if、while、return。
- FOLLOW 集中的符号。

9.4 LL 与 LR 中的错误恢复

LL 表驱动分析中，可在空表项 $M[A, a]$ 放入 synch 标记。当栈顶为非终结符 $A$ ，当前输入为 $a$ ，且 $a \in FOLLOW(A)$ 时，可弹出 $A$ ，认为缺失了某个结构。

LR 分析中，若 ACTION 表为空，可：

- 弹出状态直到某个状态可转移到错误恢复非终结符。
- 跳过输入直到同步 token。
- 移入特殊 error 符号，再继续分析。

10 综合示例：表达式文法从 LL 到 LR

原始表达式文法：

$$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow (E) \mid id.$$



它适合 LR，因为左递归表达左结合自然；但不适合递归下降。改写为 LL(1)：

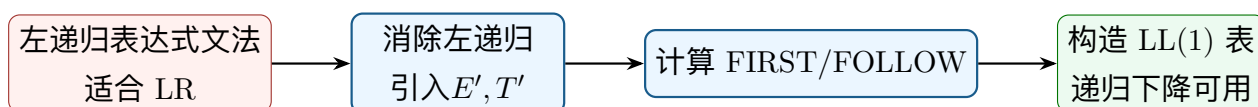
$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \varepsilon$$

$$F \rightarrow (E) \mid id$$



核心 FIRST/FOLLOW：

| 非终结符 | FIRST                  | FOLLOW              |
|------|------------------------|---------------------|
| $E$  | $\{ (, id \}$          | $\{ \$, ) \}$       |
| $E'$ | $\{ +, \varepsilon \}$ | $\{ \$, ) \}$       |
| $T$  | $\{ (, id \}$          | $\{ +, \$, ) \}$    |
| $T'$ | $\{ *, \varepsilon \}$ | $\{ +, \$, ) \}$    |
| $F$  | $\{ (, id \}$          | $\{ *, +, \$, ) \}$ |

这说明同一种语言可用不同文法服务不同 parser：LR 文法更自然表达左结合；LL 文法更适合预测分析。

## 11 复习清单

- 能写出 CFG 四元组，并区分终结符、非终结符、句型、句子、语言。
- 能解释最左推导、最右推导、parse tree、AST 和二义性。
- 能消除直接左递归、提取左因子。
- 能计算 FIRST 和 FOLLOW，并据此构造 LL(1) 表。
- 能解释递归下降、预测分析和 LL(1) 条件。
- 能解释 shift、reduce、handle、viable prefix 和冲突。
- 能构造 LR(0) 项目、CLOSURE、GOTO 和 ACTION/GOTO 表。
- 能说明 LR(0)、SLR、LR(1)、LALR 的区别和包含关系。
- 能比较 LL 与 LR 的优缺点。
- 能说明 panic-mode、phrase-level、error productions 和 global correction。

## 结语

语法分析的难点不在单个定义，而在“文法、推导、分析树、分析表、自动机状态”之间的对应关系。LL 方法强调预测：看到下一个 token 就决定展开哪个产生式；LR 方法强调证据：读入足够右部后再规约。把这两条思路对照起来，语法分析知识体系会更清晰。